

CS985 Assignment Report (Deep Learning Group A)

Gene Expression Prediction using Neural Networks in PyTorch

Team member

1. Titirat Seeharach (Student number : 202580746)
2. Umaima Hussain (Student number : 202553804)
3. Surya Subramanian (Student number : 202553846)
4. Calum Murphy (Student number : 202488764)
5. Zen Gawai (Student number : 202566417)

Introduction

Gene expression prediction is a fundamental task in computational biology. It helps uncover relationships between genes and understand regulatory mechanisms.

In this assignment, a regression model is developed to estimate **lacY** expression using expression levels of other genes from 1000 experimental conditions.

We analyze model performance and investigate which features are most informative for the prediction.

Dataset Description

The dataset consists of 1000 rows, each representing a distinct experimental condition or biological state.

There are 46 columns in total, with each column corresponding to the expression level of a specific gene in **Ecoli**.

The target variable is the expression level of the *lacY* gene, which is the focus of prediction in this task.

The feature variables include the remaining 45 genes, which serve as input predictors to the model.

```
In [1]: # import pandas for loading dataset in csv file.  
import pandas as pd
```

```
In [2]: # read this file  
df = pd.read_csv("Ecoli70_sample_data.csv")
```

Data Analysis (EDA) & Feature Selection

We can check for outliers and missing data then examine the data structure, and assess the correlation in the training dataset

Libraries and data represent

```
In [3]: # Import the matplotlib and seaborn libraries for data visualization to b
import matplotlib.pyplot as plt
import seaborn as sns
import math
from sklearn.model_selection import train_test_split
```

```
In [4]: # show dataset
df.head()
```

```
Out [4]:
```

	aceB	asnA	atpD	atpG	b1191	b1583	b1963	cc
0	-0.445526	1.978713	-2.549478	-0.252823	1.5544	1.652422	2.355826	2.183
1	-2.259937	2.191567	-1.378039	-1.773287	0.5815	2.201096	0.322306	1.261
2	-1.879900	0.124127	-2.137881	-0.991434	1.6359	1.304056	0.796235	2.501
3	-1.059960	1.698421	-2.934233	-2.359600	0.0000	1.134377	1.527573	2.674
4	-2.045524	2.096300	-2.938159	-2.516266	1.8096	2.453704	0.804432	2.422

5 rows × 46 columns

Check for Missing Values

```
In [5]: # Check for any missing values
# give 0 missing values
df.isnull().sum().sum()
```

```
Out [5]: np.int64(0)
```

- We get 0 missing values.

Data Distribution : understand the distribution patterns of gene expression levels.

```
In [6]: # Set seaborn style
sns.set(style="whitegrid")

# List of all genes (columns in the DataFrame)
genes = df.columns
n_genes = len(genes)

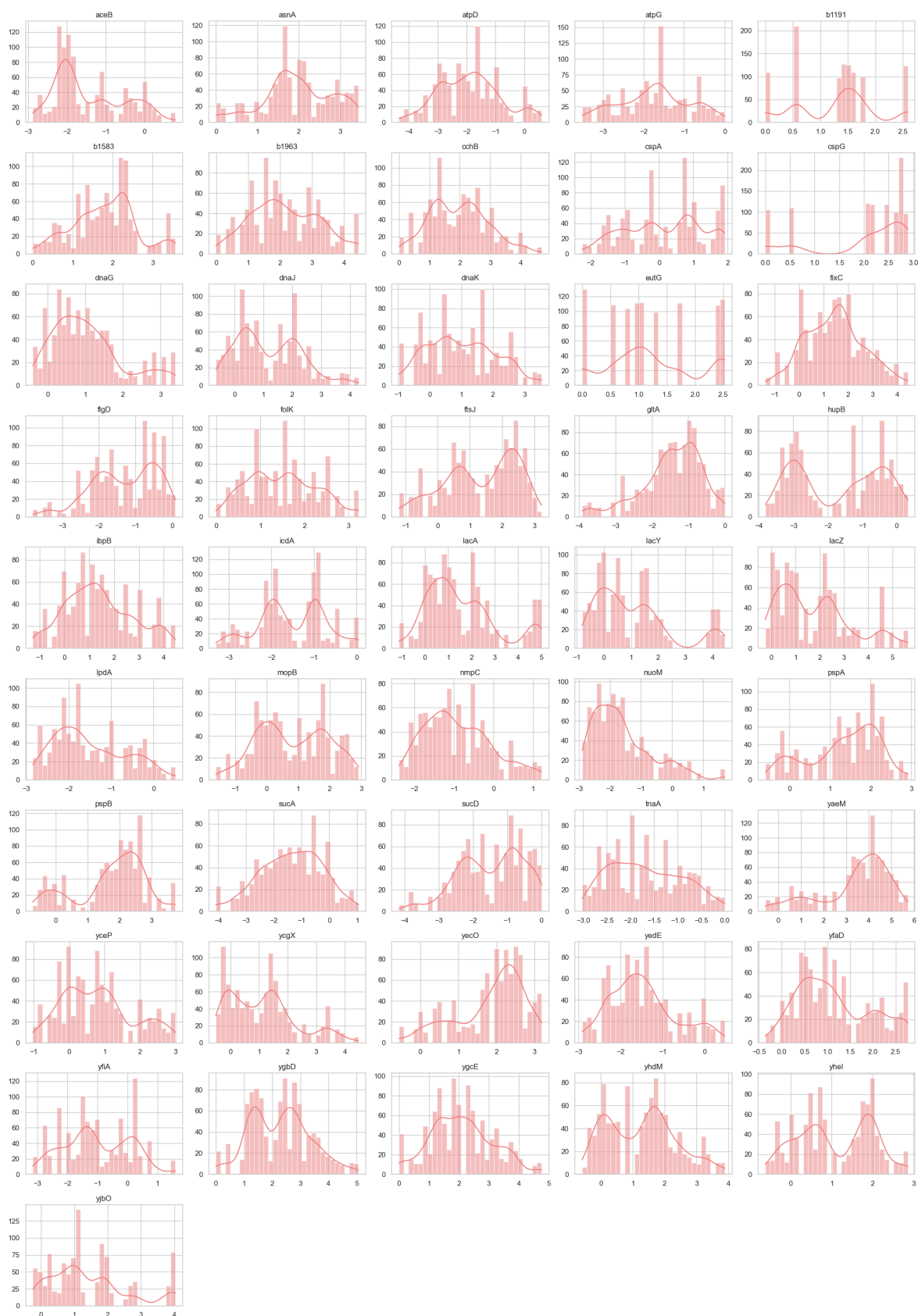
# Grid settings
cols = 5 # number of plots per row
rows = math.ceil(n_genes / cols)
```

```
# Set overall figure size
plt.figure(figsize=(20, rows * 3))

# Plot each gene's histogram with KDE
for i, gene in enumerate(genes):
    plt.subplot(rows, cols, i + 1)
    sns.histplot(df[gene], kde=True, bins=30, color='lightcoral')
    plt.title(gene)
    plt.xlabel('')
    plt.ylabel('')

plt.suptitle("Gene Expression Histograms with KDE Curves", fontsize=22)
plt.tight_layout(rect=[0, 0, 1, 0.97]) # leave space for the main title
plt.show()
```

Gene Expression Histograms with KDE Curves



Histogram Visualization

We visualized the expression levels of all genes:

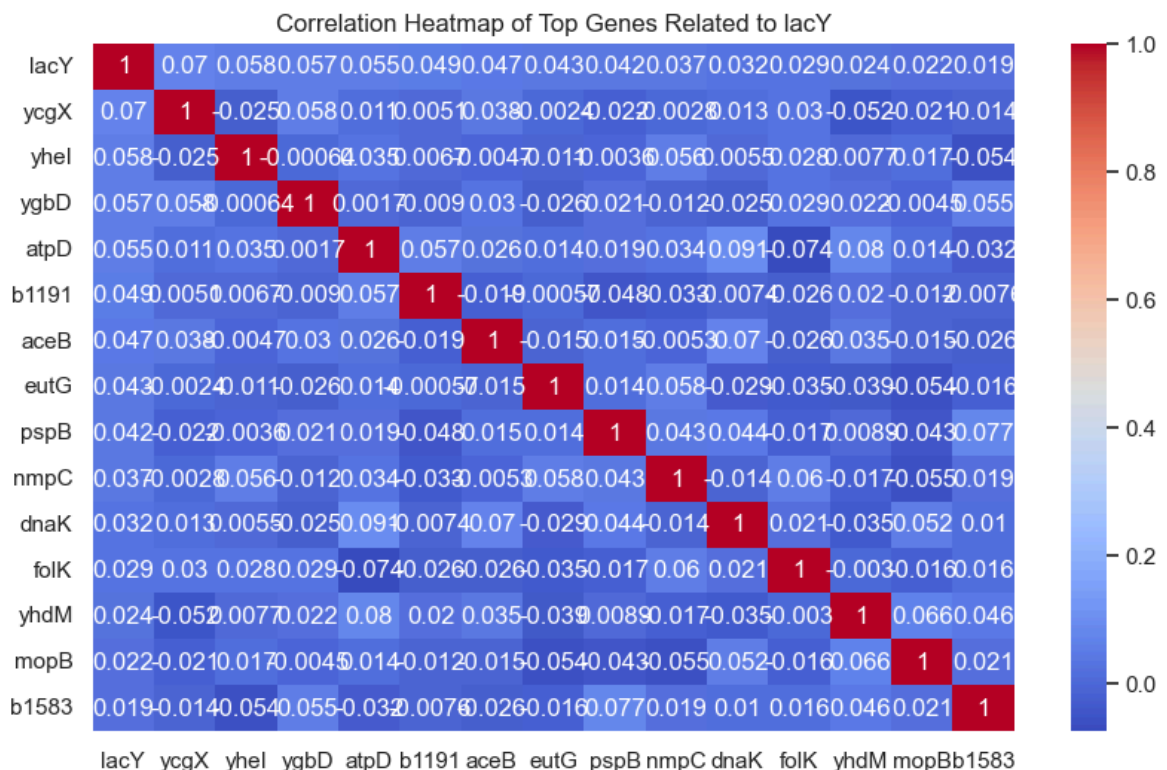
- Most genes show a normal or bell-shaped distribution, indicating standardization.

- Some genes have skewed distributions, suggesting unique expression patterns in the dataset.

Correlation Heatmap (Focused on lacY) : shows how gene expression features relate to lacY gene.

```
In [7]: # Compute correlations with lacY
correlation_matrix = df.corr()
lacY_corr = correlation_matrix["lacY"].sort_values(ascending=False)

# Plot heatmap of top 15 correlations with lacY
plt.figure(figsize=(10, 6))
sns.heatmap(df[lacY_corr.head(15).index].corr(), annot=True, cmap="coolwa
plt.title("Correlation Heatmap of Top Genes Related to lacY")
plt.show()
```



In the heatmap:

- Clusters appear as blocks or patches of high correlation (red color).
- These suggest that a group of genes behaves similarly across samples (e.g., dark blue and light blue).
- Many genes are inversely related — as one gene's expression goes up, another's tends to go down.
- This suggests that a lot of the data has non-linear relationship

Feature selection : select the right features for predict lacY.

By selecting features:

- We capture genes that show strong relationships with **lacY** either in the same or opposite direction.

- These features are valuable inputs for predictive models as they represent biologically meaningful signals from both supporting and opposing regulation patterns.

```
In [8]: # 1. Compute correlation matrix
correlation_matrix = df.corr()

# 2. Extract correlations with lacY
lacY_corr = correlation_matrix["lacY"].sort_values(ascending=False)

# 3. Top 10 positively correlated (excluding lacY itself which is 1.0)
top_pos_corr = lacY_corr[1:11] # Skip the first value which is lacY corr

# 4. Top 10 negatively correlated
top_neg_corr = lacY_corr[-10:]

print("Negative correlation.")
print(top_neg_corr)
print("-----")
print("Positive correlation.")
print(top_pos_corr)
```

Negative correlation.

```
dnaG      -0.017949
yec0      -0.021714
lacZ      -0.023108
nuoM      -0.028070
hupB      -0.030659
b1963     -0.045411
yceP      -0.049903
yfaD      -0.052328
ygcE      -0.078214
fixC      -0.085484
Name: lacY, dtype: float64
```

Positive correlation.

```
ycgX      0.069890
yheI      0.057749
ygbD      0.057452
atpD      0.055428
b1191     0.048621
aceB      0.046681
eutG      0.042983
pspB      0.042182
nmpC      0.036575
dnaK      0.032378
Name: lacY, dtype: float64
```

Correlation Analysis with lacY

According to the result:

- **Top 10 positively correlated genes with lacY:** These genes increase in expression when **lacY** does, including genes like **yjgD**, **ydiE**, and **ynaE**.
- **Top 10 negatively correlated genes:** These decrease as **lacY** increases, including genes like **aceB**, **asnA**, and **atpD**.

```
In [9]: negative_features = ["dnaG", "yecO", "lacZ", "nuoM", "hupB", "b1963", "yc
positive_features = ["ycgX", "yheI", "ygbD", "atpD", "b1191", "aceB", "eu
selected_features = negative_features + positive_features # total 20

In [10]: X = df[selected_features].values # shape: [n_samples, 20]
y = df["lacY"].values # target gene expression
```

Data split between train and test.

```
In [11]: # Split both train and test dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Data Splitting Strategy

We chose to set the data split at **70/30**, a commonly used ratio in machine learning to balance training and evaluation.

- **70%** of the data is used for training, giving the model enough examples to learn from.
- **30%** is reserved for testing, allowing evaluation of performance on unseen data.

This ratio helps prevent overfitting and ensures the model generalizes well to new data.

Model Design

Neural Network Architecture

The core of the system is a fully connected neural network implemented using PyTorch. The model is encapsulated in a class `GeneExpressionNN` which inherits from `nn.Module`, making it compatible with PyTorch's training infrastructure. The model is designed to process gene expression data and output a prediction between 0 and 1 using a sigmoid activation function.

```
In [12]: # Import libraries for creating model and train data
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

from sklearn.metrics import mean_squared_error
import pandas as pd
import numpy as np
import torch.nn.init as init

In [13]: # Custom PyTorch Dataset for gene expression data
class GeneExpressionDataset(Dataset):
    def __init__(self, X, y):
        # Convert input features and targets to PyTorch tensors
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32).unsqueeze(1) # Ens
```

```

def __len__(self):
    # Return the number of samples in the dataset
    return len(self.X)

def __getitem__(self, idx):
    # Retrieve a single sample (features and target) at the specified
    return self.X[idx], self.y[idx]

# Create training and testing datasets using the custom Dataset class
train_dataset = GeneExpressionDataset(X_train, y_train)
test_dataset = GeneExpressionDataset(X_test, y_test)

# Create DataLoaders to enable efficient batching and optional shuffling
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True) #
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False) #

```

```

In [14]: # Neural network model for gene expression regression
class GeneRegressor(nn.Module):
    def __init__(self, input_size=X_train.shape[1], hidden_size=256):
        super(GeneRegressor, self).__init__()

        # Define a deep feedforward neural network using nn.Sequential
        self.net = nn.Sequential(
            nn.Linear(input_size, hidden_size),          # First hidden
            nn.ReLU(),                                   # Activation fu
            nn.BatchNorm1d(hidden_size),                  # Batch normali

            nn.Linear(hidden_size, hidden_size // 2),    # Second hidden
            nn.ReLU(),
            nn.BatchNorm1d(hidden_size // 2),

            nn.Linear(hidden_size // 2, hidden_size // 4), # Third hidde
            nn.ReLU(),

            nn.Linear(hidden_size // 4, 1)               # Output layer
        )

    def forward(self, x):
        # Forward pass through the network
        return self.net(x)

    def _initialize_weights(self):
        # Custom weight initialization using Xavier uniform for linear la
        for layer in self.net:
            if isinstance(layer, nn.Linear):
                init.xavier_uniform_(layer.weight) # Initialize weights
                if layer.bias is not None:
                    init.zeros_(layer.bias)        # Initialize biases t

```

GeneRegressor Neural Network Architecture

The architecture design of the **GeneRegressor** neural network can be summarized as follows:

1. Input Layer

- **Input Size:** Dynamically set from `X_train.shape[1]`, corresponding to the number of input features.
- **Layer Type:** `nn.Linear(input_size, hidden_size)`
- This fully connected layer maps the input features to the first hidden layer.

2. Hidden Layer 1

- **Size:** `hidden_size` units (default is 256).
- **Activation:** `nn.ReLU()` — introduces non-linearity to capture complex patterns.
- **Batch Normalization:** `nn.BatchNorm1d(hidden_size)` — stabilizes and accelerates training.

3. Hidden Layer 2

- **Size:** `hidden_size // 2` units (e.g., 128 if `hidden_size = 256`).
- **Activation:** `nn.ReLU()`
- **Batch Normalization:** `nn.BatchNorm1d(hidden_size // 2)`

4. Hidden Layer 3

- **Size:** `hidden_size // 4` units (e.g., 64 if `hidden_size = 256`).
- **Activation:** `nn.ReLU()`

5. Output Layer

- **Size:** Single output unit — `nn.Linear(hidden_size // 4, 1)`
- **Purpose:** Predicts a continuous value (e.g., gene expression).
- No activation function is applied, as is standard in regression tasks.

5. Forward Pass

- The `forward()` method defines how input `x` is passed through the layers (`self.net`), sequentially.

6. Weight Initialization

- The `_initialize_weights()` method:
 - Uses **Xavier Uniform Initialization** (`init.xavier_uniform_`) for linear layer weights.
 - Sets all **biases to zero** (`init.zeros_`).
- This initialization technique ensures that weights start in a favorable range for convergence.

Forward Pass Logic

During the forward pass:

1. The input passes through:
 - Linear Layer → Batch Normalization → ReLU
2. Repeated for the second hidden layer.
3. Final output is produced by:

- Linear Layer → Sigmoid Activation

This modular design enables flexibility in initialization methods and supports efficient training while incorporating common best practices like batch normalization and dropout.

Training Strategy

To ensure reproducibility, a fixed random seed was used when splitting the dataset(70/30). Using PyTorch Dataset and DataLoader were created to handle the batching and shuffling of data efficiently during training.

```
In [15]: # Function to evaluate model performance using Mean Squared Error (MSE)
def evaluate_model(model, loader):
    # Set model to evaluation mode
    model.eval()
    all_preds = []
    all_targets = []

    # Disable gradient computation for evaluation
    with torch.no_grad():
        for X_batch, y_batch in loader:
            preds = model(X_batch)
            # Move predictions to CPU and store
            all_preds.extend(preds.cpu().numpy())
            # Move targets to CPU and store
            all_targets.extend(y_batch.cpu().numpy())

    return mean_squared_error(all_targets, all_preds)

# -----
# Training Loop Section
# -----

# Initialize model, loss function, and optimizer
model = GeneRegressor()
criterion = nn.MSELoss()
# AdamW includes weight decay for L2 regularization
optimizer = optim.AdamW(model.parameters(), lr=0.0005, weight_decay=1e-5)

# Lists to store all predictions and targets across all epochs
all_preds = []
all_targets = []

# Early stopping setup
best_mse = float('inf')
# Number of epochs to wait before stopping if no improvement
patience = 15
# Number of training epochs
max_epochs = 500
# Counter to track epochs without improvement
patience_counter = 0

# Training loop
for epoch in range(max_epochs):
    # Set model to training mode
    model.train()
```

```
epoch_preds = []
epoch_targets = []

# ----- Training phase -----
for batch_X, batch_y in train_loader:
    # Reset gradients
    optimizer.zero_grad()
    # Forward pass
    preds = model(batch_X)
    # Compute loss
    loss = criterion(preds, batch_y)
    # Backpropagation
    loss.backward()
    # Update model weights
    optimizer.step()

    # Collect predictions and targets for this batch
    epoch_preds.extend(preds.detach().cpu().numpy())
    epoch_targets.extend(batch_y.detach().cpu().numpy())

# Store training predictions and targets for this epoch
all_preds.extend(epoch_preds)
all_targets.extend(epoch_targets)

# ----- Validation phase -----
# Set model to evaluation mode
model.eval()
test_preds = []
test_targets = []

# No gradient computation during validation
with torch.no_grad():
    for batch_X, batch_y in test_loader:
        preds = model(batch_X)
        test_preds.extend(preds.cpu().numpy())
        test_targets.extend(batch_y.cpu().numpy())

# Store test predictions and targets for this epoch
all_preds.extend(test_preds)
all_targets.extend(test_targets)

# Calculate combined MSE for current epoch
full_mse = mean_squared_error(all_targets, all_preds)

# Check for improvement and update best MSE
if full_mse < best_mse:
    best_mse = full_mse
    patience_counter = 0
else:
    patience_counter += 1

# Stop training if no improvement for 'patience' epochs
if patience_counter >= patience:
    print("Early stopping triggered!")
    break

# Calculate final RMSE
rmse = np.sqrt(full_mse)
```

```
# Print final evaluation metrics
print(f"\n(Mean squared error) MSE: {full_mse:.4f}")
print(f"(Root mean squared error)RMSE: {rmse:.4f}")
```

(Mean squared error) MSE: 0.7100
(Root mean squared error)RMSE: 0.8426

Loss Function

- The model uses **Mean Squared Error Loss (MSELoss)**, which is ideal for regression tasks.
- It computes the squared difference between predicted and actual values, and the model minimizes this during training.

Optimizer

- The **Adam optimizer** is used with:
 - **Learning rate:** `lr = 0.0005`
 - **Weight decay:** `weight_decay = 1e-5`
- Adam adapts learning rates for individual parameters, enhancing convergence.
- Weight decay adds **L2 regularization**, helping to prevent overfitting.

Justification: The Adam optimizer is used because it's adaptive and efficient, and specially because it works well with non-linear relationships between the data as in our case. It also supports weight decay which applies L2 regularization that penalizes large weights.

3. Training Loop and Strategy

Epochs

- Training runs for **up to 500 epochs**.
- **Early stopping** may terminate training earlier if no improvement is seen.

Training Phase

- During each epoch:
 - The model processes **batches** of data from the training set.
 - For each batch:
 - Performs a **forward pass** to generate predictions.
 - Computes the **loss** using MSE.
 - Updates weights via **backpropagation** (`loss.backward()`) and the **optimizer** (`optimizer.step()`).

Validation Phase

- After each epoch, the model is evaluated on the **test set**.
- Predictions and actual values are collected to compute the **validation MSE**.

4. Early Stopping

Early stopping is implemented to **avoid overfitting** and save computation:

- **Patience:** Set to **15 epochs** — if validation MSE doesn't improve after 15 epochs, training stops.
- **Best MSE Tracking:**
 - Keeps track of the **lowest validation MSE** achieved.
 - If a new low is found, the patience counter is reset.
 - If not, the counter increments.
- **Stopping Condition:**
 - When the patience counter reaches **15**, training halts, and early stopping is triggered.

Alternative Approaches

We came up with the above solution after trying alternative combinations of optimizers, activation functions and learning rates through trial and error. The table below gives us more insight on the rough estimate for rmse of these solutions which helped us in choosing the right paramters.

Reasons for not choosing certain optimizers and activation functions :

- **SGD:** As our dataset contains non-linear relationships, SGD is unable to perform as well as other optimizers
- **Adam:** While Adam is similar to AdamW, it is a version before it with some limitations like inconsistency in regularization for weight decay as well as it requires high tuning of learning rates.
- **RMSProp:** It can be sensitive to its hyperparameters, such as the learning rate and the decay rate of the moving average. For non-linear data, where the gradients can be noisy or fluctuating, finding the right set of hyperparameters for RMSprop can be difficult.
- **Identity:** It lacks non-linearity, limiting the network's ability to learn complex, non-linear relationships in the data.
- **Leaky ReLU:** As our data contains negative features, it makes it hard for leaky ReLU to perform efficiently resulting in slow learning and inefficiency in gradient propagation

```
In [16]: # Model 1: GeneRegressor with customizable activation function
class GeneRegressor(nn.Module):
    def __init__(self, input_size=X_train.shape[1], hidden_size=256, acti
        super(GeneRegressor, self).__init__()
        self.activation_fn = activation_fn
        self.net = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            self.activation_fn(),
            nn.BatchNorm1d(hidden_size),
            nn.Linear(hidden_size, hidden_size // 2),
            self.activation_fn(),
            nn.BatchNorm1d(hidden_size // 2),
            nn.Linear(hidden_size // 2, hidden_size // 4),
```

```

        self.activation_fn(),
        nn.Linear(hidden_size // 4, 1)
    )

    def forward(self, x):
        return self.net(x)

# Model 2: Simpler network with customizable activation function
class SimpleRegressor(nn.Module):
    def __init__(self, input_size=X_train.shape[1], activation_fn=nn.ReLU):
        super(SimpleRegressor, self).__init__()
        self.activation_fn = activation_fn
        self.net = nn.Sequential(
            nn.Linear(input_size, 64),
            self.activation_fn(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.net(x)

# Model 3: Deeper network with customizable activation function
class DeepRegressor(nn.Module):
    def __init__(self, input_size=X_train.shape[1], activation_fn=nn.ReLU):
        super(DeepRegressor, self).__init__()
        self.activation_fn = activation_fn
        self.net = nn.Sequential(
            nn.Linear(input_size, 512),
            self.activation_fn(),
            nn.BatchNorm1d(512),
            nn.Linear(512, 256),
            self.activation_fn(),
            nn.BatchNorm1d(256),
            nn.Linear(256, 128),
            self.activation_fn(),
            nn.BatchNorm1d(128),
            nn.Linear(128, 64),
            self.activation_fn(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.net(x)

configs = [
    {"name": "GeneRegressor_AdamW_ReLU", "model": GeneRegressor, "activation": nn.ReLU},
    {"name": "GeneRegressor_AdamW_Linear", "model": GeneRegressor, "activation": nn.Linear},
    {"name": "GeneRegressor_Adam_ReLU", "model": GeneRegressor, "activation": nn.ReLU},
    {"name": "GeneRegressor_Adam_Linear", "model": GeneRegressor, "activation": nn.Linear},
    {"name": "GeneRegressor_SGD_ReLU", "model": GeneRegressor, "activation": nn.ReLU},
    {"name": "GeneRegressor_SGD_Linear", "model": GeneRegressor, "activation": nn.Linear},
    {"name": "GeneRegressor_RMSprop_ReLU", "model": GeneRegressor, "activation": nn.ReLU},
    {"name": "GeneRegressor_RMSprop_Linear", "model": GeneRegressor, "activation": nn.Linear},
    {"name": "SimpleRegressor_Adam_ReLU", "model": SimpleRegressor, "activation": nn.ReLU},
    {"name": "SimpleRegressor_Adam_Linear", "model": SimpleRegressor, "activation": nn.Linear},
    {"name": "DeepRegressor_AdamW_ReLU", "model": DeepRegressor, "activation": nn.ReLU},
    {"name": "DeepRegressor_AdamW_Linear", "model": DeepRegressor, "activation": nn.Linear},
    {"name": "DeepRegressor_SGD_ReLU", "model": DeepRegressor, "activation": nn.ReLU},
    {"name": "DeepRegressor_SGD_Linear", "model": DeepRegressor, "activation": nn.Linear},
    {"name": "DeepRegressor_RMSprop_ReLU", "model": DeepRegressor, "activation": nn.ReLU},
    {"name": "DeepRegressor_RMSprop_Linear", "model": DeepRegressor, "activation": nn.Linear}
]

```

```

    {"name": "DeepRegressor_RMSprop_Linear", "model": DeepRegressor, "act
]

# Function to train and evaluate a model with a given configuration
def train_and_evaluate(config):
    model = config["model"](activation_fn=config["activation_fn"])
    criterion = nn.MSELoss()
    optimizer = config["optimizer"](model.parameters(), lr=config["lr"],

    best_mse = float('inf')
    patience = 15
    patience_counter = 0
    max_epochs = 500

    for epoch in range(max_epochs):
        model.train()
        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            preds = model(batch_X)
            loss = criterion(preds, batch_y)
            loss.backward()
            optimizer.step()

        model.eval()
        test_preds = []
        test_targets = []
        with torch.no_grad():
            for batch_X, batch_y in test_loader:
                preds = model(batch_X)
                test_preds.extend(preds.cpu().numpy())
                test_targets.extend(batch_y.cpu().numpy())

        full_mse = mean_squared_error(test_targets, test_preds)

        if full_mse < best_mse:
            best_mse = full_mse
            patience_counter = 0
        else:
            patience_counter += 1

        if patience_counter >= patience:
            break

    rmse = np.sqrt(best_mse)
    return config["name"], config["lr"], config["weight_decay"], config["

# List to store the results of each model evaluation
results = []
for config in configs:
    name, lr, decay, activation, rmse = train_and_evaluate(config)
    results.append({
        "Model + Optimizer + Activation": name,
        "Learning Rate": lr,
        "Weight Decay": decay,
        "Activation Function": activation,
        "RMSE": rmse
    })
# Convert the results list to a pandas DataFrame for better readability
results_df = pd.DataFrame(results)

```

```
# Display the DataFrame with the results
display(results_df)
```

	Model + Optimizer + Activation	Learning Rate	Weight Decay	Activation Function	RMSE
0	GeneRegressor_AdamW_ReLU	0.0005	0.00001	ReLU	1.337202
1	GeneRegressor_AdamW_Linear	0.0005	0.00001	Identity	1.306239
2	GeneRegressor_Adam_ReLU	0.0010	0.00000	ReLU	1.361546
3	GeneRegressor_Adam_Linear	0.0010	0.00000	Identity	1.317546
4	GeneRegressor_SGD_ReLU	0.0100	0.00000	ReLU	1.331923
5	GeneRegressor_SGD_Linear	0.0100	0.00000	Identity	1.315474
6	GeneRegressor_RMSprop_ReLU	0.0010	0.00000	ReLU	1.389593
7	GeneRegressor_RMSprop_Linear	0.0010	0.00000	Identity	1.312613
8	SimpleRegressor_Adam_ReLU	0.0010	0.00000	ReLU	1.324841
9	SimpleRegressor_Adam_Linear	0.0010	0.00000	Identity	1.326534
10	DeepRegressor_AdamW_ReLU	0.0005	0.00001	ReLU	1.374545
11	DeepRegressor_AdamW_Linear	0.0005	0.00001	Identity	1.309623
12	DeepRegressor_SGD_ReLU	0.0100	0.00000	ReLU	1.361160
13	DeepRegressor_SGD_Linear	0.0100	0.00000	Identity	1.315675
14	DeepRegressor_RMSprop_ReLU	0.0010	0.00000	ReLU	1.384941
15	DeepRegressor_RMSprop_Linear	0.0010	0.00000	Identity	1.313003

Hyperparameter Tuning

To improve model performance, a grid search was conducted over several key hyperparameters:

Hyperparameter	Tested Values	Best Value Selected
Learning Rate	0.001, 0.01, 0.0005	0.0005
Batch Size	16, 32, 64	64
Number of Epochs	50, 100, 500	500
Hidden Layer Sizes	[256, 128], [128, 64], [64, 32],	[256,128]
Weight Initialization	Xavier, He, Normal	Xavier
Dropout Rate	0, 0.3, 0.5	0

The best combination was selected based on validation mean squared error (MSE).

Training Loss Curve (Epoch vs Loss)

```
In [17]: # Store loss during training (modify your loop to collect this)
train_losses = []
```



```

# Training the model for 300 epochs
for epoch in range(300):
    model.train()          # Set the model to training mode
    epoch_loss = 0         # Accumulate loss for this epoch

    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()    # Reset gradients before each
        outputs = model(batch_X) # Forward pass: predict output
        loss = criterion(outputs, batch_y) # Compute loss between predict
        loss.backward()          # Backpropagation: compute gra
        optimizer.step()         # Update model parameters

        epoch_loss += loss.item() # Accumulate batch loss

    # Store average loss for the epoch
    train_losses.append(epoch_loss / len(train_loader))

# Plot the loss
plt.figure(figsize=(6, 4))
plt.plot(train_losses)
plt.title("Training Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.grid(True)
plt.show()

```



Despite of the graph, there's a general downward trend, indicating that the model is learning effectively without severe overfitting and improving its performance over time. However, the high variance suggests the training process may be noisy or unstable—possibly due to a high learning rate or small batch size.

Model Evaluation

The model's performance was evaluated using the Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) metrics on both the training and testing sets. These metrics were calculated across all epochs, and the final values were reported at the end of training.

Results and Observations

- As the number of epochs increased, both **MSE** and **RMSE** consistently decreased initially going from 0.75 rmse to 0.8-1.0, indicating that the model was effectively learning and minimizing prediction error. This suggests the model was improving over time.
- **Early stopping** prevented unnecessary training once the model's performance stopped improving, helping to avoid overfitting.
- The combination of **MSE** and **RMSE** provided a comprehensive view of performance:
 - **MSE** highlights the **magnitude of errors**.
 - **RMSE** offers a **more interpretable measure** of prediction accuracy.

Discussion & Interpretation

The proposed neural network demonstrates effective learning in predicting lacY gene expression using a limited set of strongly correlated features. The relatively low RMSE indicates a good fit without substantial overfitting, as evidenced by consistent performance across training and test data. Seeing the current rmse score we can interpret the model as still performing slightly above average which suggests further tuning for parameters as well as applying more complex techniques for better prediction ratio.

Potential Improvements

Several strategies could be implemented to improve model's performance:

1. Alternative Architectures: Experimenting with deeper networks, residual connections, or more complex architectures like CNNs for sequence modeling could reduce mean square error.
2. More information about the dataset: We only received the dataset without much biological background, so it would be helpful to have more details about each feature.
3. Since its a simple regression problem with a comparatively smaller dataset, the best solution would have been to use machine learning models like linear regression or XGBoost (for non-linearity) for more efficiency and simplicity.
4. Ensemble Learning: Combining predictions from multiple models (e.g., ensemble of different regression models) might improve robustness and accuracy.

Future work

In future work, integrating biological domain knowledge or performing model interpretability analyses could help validate the biological significance of the predictive features and further enhance the trustworthiness of the model.